

CS 161 Shapes Lab

Problem Description

For this assignment, you will write a program that reads from one file, interprets what it reads, and writes to a second file. For example, if it reads **S * 3**, it will create a **square** (**S** is **code** for square) made of the *** symbols** which is **3 by 3**.

The program should use the functions described below exactly as specified. This program includes these concepts: decisions, loops, menus, functions, strings, lists, reading from a file, and writing to a file.

Two Flags in Moodle

There are two flags for this assignment with two corresponding code checkers. To capture the first flag, you will create a driver and stubs in separate files, save the files in a folder, zip the folder, and send it to the **LISTShapesStubs** code checker. To capture the second flag, you will get the full program working, save the files in a folder, zip the folder, and send it to the **LISTShapesFull** code checker.

In both cases, your program should contain these files.

- **main.py** – **header comment** at top of file which includes a brief program **description**, your **name**, and the **date**; **main** function definition
- **shapes.py** – **function definitions** with a **docstring** comment for each function

To test your program, you will need an input file called **input**. Place it in the same folder with your .py files. At the end of these instructions, there is example content for this file, so you can see how it should be formatted. You do not need to include the input file in the folder you submit. The code checkers will use a different input file.

Main Program Functionality

Your **main** function should have the following functionality:

- **Open** a file named **input** for reading. If the file **cannot be opened**, **report** an **error** and **exit** the program. Test for this by temporarily removing (or renaming) the input file in the folder where your program will look for files to read. This should be the same folder which contains main.py.
- **Open** a file named **output** for writing.

- **Read** the information in **input**, one line at a time using a loop, **until** an **E** is read or the **end of file** is reached, in which case the program should **terminate**. **For each line**, do the following.
 - Set a variable to the code at the beginning of the line and another variable to the rest of the line. The latter variable will be a list. For example, **if** the **line** contains **S * 3**, then the **first variable** will be the **'S'** and the **second** will be **['*', '3']**. You'll send the elements in the list as arguments to the functions you'll define in shapes.py. The functions will require numbers, so you'll need to convert the arguments to integers with **int** before sending them in your function calls.
 - Use the **code** you read at the beginning of the line to **determine** which **shape** drawing **function** you should call. If the code **doesn't match E, R, S, or T**, you should report "**Invalid Code**" to the screen.
 - **Call** the appropriate **shape drawing function** and **pass** it the **integer arguments** it needs. See sections below for more details.
 - **Save** the **string returned** by the shape drawing function.
 - **Call writeShape**, and **pass** it the **string** along with the **reference to** your **output file**.
- **Close** both **files**.

Interpreting lines in infile

Except for lines starting with E, each line in input will have the following format.

code symbol value1 value2 – where **value2** is **optional**

- The **code** will be a single character from this list: **R, S, T, E**.
- The **symbol** will be a single character. It will be used to make the **output shape**.
- For **S** and **T**, there will be a **single integer** value indicating the **shape's dimension**.
- For **R**, there will be **two integer values** indicating the **shape's dimensions**.
- For **E**, there will be **nothing** to read **after** the **E**.

Required Functions

The functions as specified below are required:

- **makeRectangle**
 - three **parameters** – **symbol**, **length**, **width** (e.g., *, 3, 2)
 - returns a string (e.g., "***\n***")
- **makeSquare**
 - two **parameters** – **symbol**, **side** (e.g., +, 3)
 - returns a string (e.g., "+++\\n+++\\n+++")
- **makeTriangle**
 - two **parameters** – **symbol**, **height** (e.g., @, 4)
 - returns a string (e.g., "@\\n@@\\n@@@\\n@@@@")
- **writeShape**
 - two **parameters** – string containing a **shape specification**, reference parameter for an **output file**
 - writes the shape string to the output file
 - returns nothing

Codes and Functions

The codes and their meanings are as follows:

End

If the code is an **E**, **end** the **program**.

Square

If the code is an **S**, create a **square** comprised of the symbol. For example, if the line contains the following,

S ! 3

then create a square which looks like this.

!!!

!!!

!!!

If the line contains the following,

S @ 5

then create a square which looks like this.

@@@@@

@@@@@

@@@@@

@@@@@

@@@@@

Triangle

If the code is a **T**, create a **triangle** comprised of the symbol. For example, if the line contains the following,

T = 3

then create a triangle which looks like this.

=

==

===

If the line contains the following,

T ^ 5

then create a triangle which looks like this.

^

^^

^^^

^^^^

^^^^^

Rectangle

If the code is an **R**, create a **rectangle** comprised of the symbol. Rectangles have two values: **rows** and **columns**. If the line contains the following,

R a 2 6

then create a rectangle which looks like this.

aaaaaa

aaaaaa

If the line contains the following,

R & 4 2



then create a rectangle which looks like this.

&&

&&

&&

&&

Example Input File

T & 4

S @ 6

T x 5

R * 5 7

R = 4 3

E

Corresponding Output File

&

& &

& & &

& & & &

@ @ @ @ @ @

@ @ @ @ @ @

@ @ @ @ @ @

@ @ @ @ @ @

@ @ @ @ @ @

@ @ @ @ @ @

x

xx

xxx



XXXX

XXXXXX

* * * * *

* * * * *

* * * * *

* * * * *

* * * * *

===

===

===

===